06 — Segurança

Versão 1.0 · atualizado em 08/11/2026

Este documento define a autenticação e a autorização do **Continuum Control** (§22): como o usuário entra, o que cada perfil pode fazer, as regras que dependem do recurso manipulado e como o acesso de quem sai é bloqueado. As regras `S1` … `S7` são citadas na coluna "Acesso" do doc 05; `D` e `A` continuam a numeração dos outros documentos.

Tecnologia	Spring Security 6 com *OAuth2 Resource Server* (JWT assinado com HMAC-SHA256) e BCrypt
Documentos relacionados	00 — Enunciado · 01 — Modelo de domínio · 04 — Persistência · 05 — API

Sumário

1. Visão geral
2. Autenticação
3. A cada requisição
4. Perfis
5. Matriz de permissões
6. Regras de segurança
7. Implementação no Spring Security
8. Senhas e segredos
9. Usuários iniciais
10. Testes
11. Decisões de modelagem
12. Pontos em aberto
13. Histórico de revisões

1. Visão geral

A autorização acontece em três camadas, cada uma com uma pergunta:

Camada	Pergunta	Falha
Autenticação (filtro do Spring Security)	Quem é? O token é válido e o usuário está ativo?	`401`
Perfil (`@PreAuthorize` no controller)	Este perfil pode chamar esta rota?	`403`
Posse (serviço)	Este usuário pode mexer **neste** recurso? (§22: "possuir perfil de agente não significa necessariamente possuir permissão para alterar qualquer missão")	`403`

Depois das três vêm as guardas do domínio (doc 03), que respondem `409`.

2. Autenticação

1. O cliente envia `login` e `senha` para `POST /auth/login` (doc 05, seção 4).
2. O serviço busca o usuário pelo login e confere a senha com o BCrypt. Se o login não existir, se a senha não conferir ou se o usuário estiver desativado, a resposta é a mesma: `401` com "Login ou senha inválidos" (S2).
3. Se tudo confere, a API emite um JWT e o devolve com o instante de expiração.
4. Nas requisições seguintes, o cliente envia `Authorization: Bearer <token>`.

O token:

Item	Valor
Algoritmo	`HS256` (HMAC com SHA-256)
Segredo	`JWT_SECRET`, com pelo menos 32 bytes aleatórios, só em variável de ambiente (§24)
Validade	60 minutos (`continuum.jwt.validade`)
`sub`	`id` do usuário
`iss`	`continuum-control`
`perfil`, `agenteId`	Informativos, para o cliente montar a tela. **A autorização não usa esses valores** (D55)
`iat`, `exp`	Emissão e expiração

Não há *refresh token*: quando o token expira, o cliente faz login de novo (D53).

3. A cada requisição

1. O filtro do *Resource Server* confere a assinatura, o emissor e a expiração do token. Se qualquer um falhar: `401`.
2. Um conversor carrega o usuário do PostgreSQL pelo `sub`, uma busca pela chave primária. Se o usuário não existir ou estiver desativado: `401` (S7).
3. A autenticação é montada com os dados **do banco**: `id`, `perfil` (como `ROLE_<PERFIL>`) e `agenteId`.

Assim, desativar um usuário ou mudar o seu perfil vale a partir da requisição seguinte, sem esperar o token expirar (fecha A11, D54).

4. Perfis

Perfil	Papel na Continuum Authority	O que faz
`OPERADOR`	Conduz as operações do dia a dia	Cadastra e altera eventos e anomalias, altera o risco, planeja missões (cria, edita, designa e remove agentes), inicia, cancela, conclui e registra falha
`AGENTE`	Agente temporal em campo	Consulta, e registra intervenções e ocorrências **nas missões em que está designado** (S3)
`SUPERVISOR`	Responde pelas operações de risco	Tudo do operador, mais: autorizar missões (§8), declarar anomalias irreversíveis, remover eventos e consultar o histórico dos agentes
`ADMINISTRADOR`	Gestão de pessoas e acessos	Tudo do supervisor, mais: gerenciar agentes e usuários

Todo perfil autenticado consulta agentes, eventos, anomalias, missões, intervenções, ocorrências e indicadores. As exceções são os usuários (só `ADMINISTRADOR`) e o histórico dos agentes (`SUPERVISOR` e `ADMINISTRADOR`).

Intervenções e ocorrências são registradas só pelo perfil `AGENTE`: são ações do agente em campo (§10, §11), e a FK do banco exige que o autor seja um agente designado.

5. Matriz de permissões

Gerada a partir da coluna "Acesso" do doc 05. ✓ = permitido; "posse" = permitido só se a regra S3 também for atendida.

Grupo	Rota	OPERADOR	AGENTE	SUPERVISOR	ADMINISTRADOR
Autenticação e usuários	`POST /auth/login`	✓ sem login	✓ sem login	✓ sem login	✓ sem login
	`GET /auth/eu`	✓	✓	✓	✓
	`PUT /auth/senha`	✓	✓	✓	✓
	`POST /usuarios`				✓
	`GET /usuarios`				✓
	`GET /usuarios/{id}`				✓
	`PATCH /usuarios/{id}`				✓
	`POST /usuarios/{id}/desativar`				✓
	`POST /usuarios/{id}/reativar`				✓
	`POST /usuarios/{id}/redefinir-senha`				✓
Agentes	`POST /agentes`				✓
	`GET /agentes`	✓	✓	✓	✓
	`GET /agentes/{id}`	✓	✓	✓	✓
	`PATCH /agentes/{id}`				✓
	`POST /agentes/{id}/suspender`				✓
	`POST /agentes/{id}/reativar`				✓
	`POST /agentes/{id}/inativar`				✓
	`GET /agentes/{id}/historico`			✓	✓
Eventos históricos	`POST /eventos`	✓		✓	✓
	`GET /eventos`	✓	✓	✓	✓
	`GET /eventos/{id}`	✓	✓	✓	✓
	`PATCH /eventos/{id}`	✓		✓	✓
	`DELETE /eventos/{id}`			✓	✓
Anomalias	`POST /anomalias`	✓		✓	✓
	`GET /anomalias`	✓	✓	✓	✓
	`GET /anomalias/{id}`	✓	✓	✓	✓
	`PATCH /anomalias/{id}`	✓		✓	✓
	`POST /anomalias/{id}/alterar-risco`	✓		✓	✓

Grupo	Rota	OPERADOR	AGENTE	SUPERVISOR	ADMINISTRADOR
	`POST /anomalias/{id}/declarar-irreversivel`			✓	✓
	`GET /anomalias/{id}/historico`	✓	✓	✓	✓
Missões	`POST /missoes`	✓		✓	✓
	`GET /missoes`	✓	✓	✓	✓
	`GET /missoes/{id}`	✓	✓	✓	✓
	`PATCH /missoes/{id}`	✓		✓	✓
	`POST /missoes/{id}/designacoes`	✓		✓	✓
	`DELETE /missoes/{id}/designacoes/{agenteId}`	✓		✓	✓
	`POST /missoes/{id}/autorizar`			✓	✓
	`POST /missoes/{id}/iniciar`	✓		✓	✓
	`POST /missoes/{id}/cancelar`	✓		✓	✓
	`POST /missoes/{id}/concluir`	✓		✓	✓
	`POST /missoes/{id}/registrar-falha`	✓		✓	✓
Intervenções e ocorrências	`POST /missoes/{id}/intervencoes`		posse		
	`GET /missoes/{id}/intervencoes`	✓	✓	✓	✓
	`PATCH /missoes/{id}/intervencoes/{intervencaoId}`		posse		
	`POST /missoes/{id}/ocorrencias`		posse		
	`GET /missoes/{id}/ocorrencias`	✓	✓	✓	✓
	`GET /missoes/{id}/ocorrencias/{ocorrenciaId}`	✓	✓	✓	✓
Indicadores	`GET /estado-operacional`	✓	✓	✓	✓
	`GET /indice-estabilidade`	✓	✓	✓	✓

6. Regras de segurança

#	Regra	Onde	Falha
S1	Toda rota exige autenticação, exceto `POST /auth/login` e `GET /actuator/health`	Filtro	`401`
S2	O login não revela se o usuário existe, se a senha errou ou se ele está desativado: a mensagem é sempre a mesma	Serviço de autenticação	`401`
S3	**Posse da missão:** o usuário `AGENTE` só registra intervenção ou ocorrência numa missão em que o seu agente tem participação. Só altera o resultado de uma intervenção que ele mesmo registrou	Serviço, antes das guardas do domínio	`403`
S4	**Identidade só do token:** responsável pela autorização, autor dos registros e agente da intervenção ou ocorrência vêm do usuário autenticado; o corpo da requisição não tem esses campos	Serviço	—
S5	Sempre há pelo menos um `ADMINISTRADOR` ativo: não é possível desativar o último, nem trocar o seu perfil. Ninguém desativa a si mesmo	Serviço de usuários	`409`
S6	Inativar um agente desativa o usuário ligado a ele (doc 03, operação 6.5). Um usuário ligado a um agente `INATIVO` não pode ser reativado	Serviços de agente e de usuário	`409`

#	Regra	Onde	Falha
S7	Usuário desativado ou removido do banco perde o acesso na requisição seguinte	Conversor do token (seção 3)	`401`

Ordem das conferências em S3: primeiro a posse (`403`), depois as guardas do domínio (`409`). Um agente que nunca foi designado recebe `403`. Um agente designado numa missão que já terminou recebe `409` ("só missões em execução recebem intervenções", M9), porque a participação dele existe.

7. Implementação no Spring Security

Trecho de referência (Spring Boot 3, Spring Security 6):

```
@Configuration
@EnableMethodSecurity
class SegurancaConfig {

  @Bean
  SecurityFilterChain api(HttpSecurity http, UsuarioDoToken conversor,
                          ProblemaNaoAutenticado entrada, ProblemaSemPermissao negado) throws
Exception {
    return http
        .csrf(csrf -> csrf.disable())                                    // API sem cookies (D53)
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.POST, "/auth/login").permitAll()  // S1
            .requestMatchers("/actuator/health/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o
            .jwt(j -> j.jwtAuthenticationConverter(conversor))           // seção 3
            .authenticationEntryPoint(entrada))                          // 401 em ProblemDetail
        .exceptionHandling(e -> e.accessDeniedHandler(negado))           // 403 em ProblemDetail
        .build();
  }

  @Bean JwtDecoder jwtDecoder(@Value("${continuum.jwt.segredo}") String segredo) {
    return NimbusJwtDecoder.withSecretKey(chave(segredo)).macAlgorithm(MacAlgorithm.HS256).build();
  }

  @Bean JwtEncoder jwtEncoder(@Value("${continuum.jwt.segredo}") String segredo) {
    return new NimbusJwtEncoder(new ImmutableSecret<>(chave(segredo)));
  }

  @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(10); }
}
```

- **`UsuarioDoToken`** implementa `Converter<Jwt, AbstractAuthenticationToken>`: carrega o usuário pelo `sub` e lança `DisabledException` se ele não existir ou estiver desativado. Essa exceção é uma `AuthenticationException`, então o *entry point* responde `401`.
- **Perfil por rota:** `@PreAuthorize("hasAnyRole('OPERADOR','SUPERVISOR','ADMINISTRADOR')")` nos métodos dos controllers, conforme a coluna "Acesso" do doc 05.
- **Posse (S3):** um componente `PosseDeMissao` chamado no início dos serviços de intervenção e ocorrência. Ele consulta `missao_agente` pelo par (missão, agente do usuário) e lança `AccessDeniedException` se não houver participação.
- **Usuário autenticado no serviço:** um objeto `UsuarioAutenticado(id, perfil, agenteId)` é o *principal* da autenticação e chega aos controllers com `@AuthenticationPrincipal`.

- **Cuidado com o tratamento global de erros:** um `@RestControllerAdvice` que capture `Exception` também captura `AccessDeniedException` e `AuthenticationException`, e responderia `500`. Ele deve tratá-las explicitamente (`403` e `401`) ou deixá-las passar para o Spring Security.

O trecho mostra a estrutura, mas não foi compilado nesta revisão: os testes da seção 10 são a verificação.

8. Senhas e segredos

- Senhas com BCrypt, custo 10, entre 8 e 72 caracteres. O BCrypt ignora o que passa de 72 bytes, por isso o limite.
- `senha_hash` nunca aparece em resposta nem em log.
- Tokens, senhas e o `JWT_SECRET` nunca são registrados em log.
- O segredo do JWT e as senhas dos bancos vêm só de variáveis de ambiente (doc 04, seção 7). Trocar o `JWT_SECRET` invalida todos os tokens emitidos.
- **CSRF desligado:** a API não usa cookies; o token vai no cabeçalho, que um site de terceiros não consegue enviar sozinho.
- **HTTPS:** em produção, terminado no *proxy* reverso à frente da aplicação. Sem HTTPS, o token pode ser capturado na rede.

9. Usuários iniciais

O §25 pede dados carregados no ambiente de desenvolvimento ou avaliação. Os usuários de teste ficam num carregador `@Profile("dev")` (um `ApplicationRunner`), que gera os hashes com o próprio `PasswordEncoder` e só roda quando a tabela `usuario` está vazia. Ele nunca roda em produção.

Login	Perfil	Agente vinculado	Senha
`admin`	`ADMINISTRADOR`	—	`continuum-dev`
`supervisor`	`SUPERVISOR`	—	`continuum-dev`
`operador`	`OPERADOR`	—	`continuum-dev`
`ana.lima`	`AGENTE`	`AG-001`	`continuum-dev`
`bruno.reis`	`AGENTE`	`AG-002`	`continuum-dev`

O README do repositório deve listar essa tabela, para que quem avalia consiga entrar com cada perfil (§26).

10. Testes

Cenários mínimos de segurança (§23), além dos testes de cada regra do domínio:

Cenário	Esperado
Rota protegida sem token	`401`
Token com assinatura adulterada, expirado ou de outro emissor	`401`
Usuário desativado com um token ainda válido	`401` (S7)
Login com senha errada e login de usuário desativado	`401` com a mesma mensagem (S2)
Cada perfil em cada rota da matriz da seção 5	`403` onde não há ✓
Agente registra intervenção em missão onde não está designado	`403` (S3)

Cenário	Esperado
Agente designado registra intervenção em missão já concluída	`409` (M9)
Corpo com `agenteId` ou `responsavelId`	O campo é ignorado; vale o usuário do token (S4)
Desativar o último administrador; desativar a si mesmo	`409` (S5)
Inativar um agente com usuário	O usuário fica desativado (S6)
Trocar o perfil de um usuário com um token já emitido	A requisição seguinte usa o perfil novo (D55)

A matriz da seção 5 pode virar um teste parametrizado: uma linha por rota e perfil, com o status esperado.

11. Decisões de modelagem

D53 — JWT sem estado e sem *refresh token*

A API não guarda sessão: qualquer instância atende qualquer requisição, e o token vai no cabeçalho. A validade de 60 minutos com novo login dispensa *refresh tokens*, que exigiriam guardá-los e revogá-los.

- *Descartada*: sessão no Redis (Spring Session), que derrubaria o acesso na hora ao desativar, mas daria ao Redis uma segunda responsabilidade (doc 04, D42). A S7 consegue o mesmo efeito.
- *Descartada*: HTTP Basic, que enviaria a senha em toda requisição.

D54 — O usuário é carregado do banco a cada requisição (fecha A11)

Uma busca pela chave primária por requisição custa pouco e garante que desativação e mudança de perfil valham na hora (S7).

- *Descartada*: confiar só no token, o que deixaria um usuário desativado operando até o token expirar.
- *Descartada*: lista de tokens revogados no Redis, que acrescentaria uma dependência ao caminho de toda requisição.

D55 — A autorização usa o perfil do banco, não o do token

O `perfil` no token serve só ao cliente. Se a autorização o usasse, rebaixar um usuário só valeria quando o token expirasse.

D56 — Posse antes das guardas do domínio

A posse (S3) é conferida antes das guardas de estado, para que um agente estranho à missão nunca receba `409`, que revelaria detalhes do estado de uma missão em que ele não atua.

D57 — Sempre um administrador ativo

Sem a S5, o último administrador poderia se desativar ou perder o perfil, e ninguém mais conseguiria gerenciar usuários sem mexer direto no banco.

12. Pontos em aberto

Fechados neste documento

#	Questão	Decisão
A11	Como bloquear o acesso de quem sai	`usuario.ativo` conferido a cada requisição (S7, D54)

Novos

#	Questão	Proposta atual	Fechar em
A17	Bloquear o login após várias senhas erradas?	Fora do escopo do trabalho; registrar como melhoria	Confirmar com o grupo

13. Histórico de revisões

Versão	Data	Alteração
1.0	2026-11-08	Versão inicial: JWT, carregamento do usuário a cada requisição, perfis, matriz de permissões gerada a partir do doc 05, regras S1–S7, usuários de desenvolvimento, testes; D53–D57; fecha A11